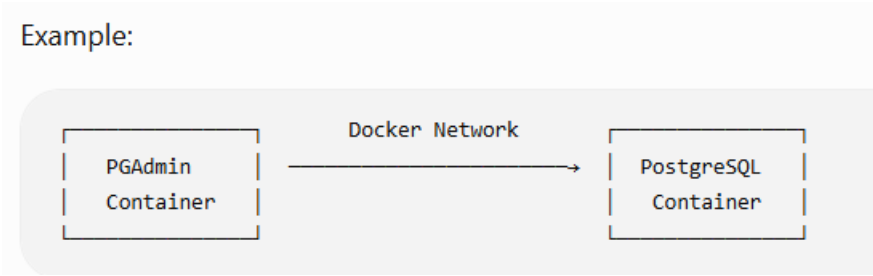
Docker Networks – Hands-on Demonstration

1. What is a Docker Network?

- A **Docker network** allows **containers** to **communicate with each other.**
- This is important when an **application** consist of **multiple container.**
- **Containers** connected to the **same Docker network** can **communicate** with each other using the **container names as hostnames**



2. Example used in this lecture

Container 1 – PostgreSQL

- Runs PostgreSQL database

Container 2 – pgAdmin

- runs **pgAdmin**
- **pgAdmin** is a web-based **PostgreSQL Management Tool**
- It provides a UI through which we can
  - Connect to database
  - Query database
  - Create database
  - Create tables
  - Manage PostgreSQL

3. Create a Docker Network

- Creating a docker network **does’nt automatically connect to** existing container**.**

Command:

‹/› **Bash**

```
docker network create my-network
```

This creates a Docker network called:

```
my-network
```

4. Start PostgreSQL with the Network

Bash

```
docker run -d --name db \
  --network my-network \
  -e POSTGRES_PASSWORD=<password> \
  postgres:14
```

Understand the options

```
docker run
    ↓
Create and run container

-d
    ↓
Detached/background mode

--name db
    ↓
Container name = db

-e
    ↓
Set environment variable

POSTGRES_PASSWORD
    ↓
Set PostgreSQL password

postgres:14
    ↓
PostgreSQL Docker image, version 14
```

5. Start pgAdmin with the same network

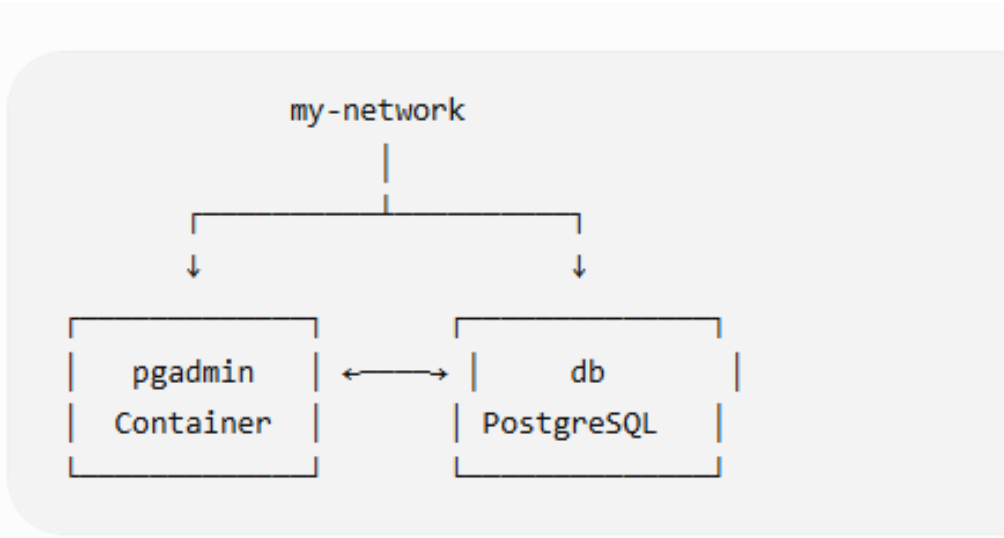
Bash

```
docker run -d --name pgadmin \
  --network my-network \
  -e PGADMIN_DEFAULT_EMAIL=user@domain.com \
  -e PGADMIN_DEFAULT_PASSWORD=<password> \
  dpage/pgadmin4
```

Important

- `PGADMIN_DEFAULT_EMAIL` → default pgAdmin login email
- `PGADMIN_DEFAULT_PASSWORD` → default pgAdmin password
- `dpage/pgadmin4` → pgAdmin Docker image

6. Docker Network



Both containers belong to the **same Docker network**.

7. Test Communication again

Bash

```
docker exec -it pgadmin ping db
```

What does this mean?

```
docker exec
    ↓
Execute a command inside a container

-it
    ↓
Interactive terminal

pgadmin
    ↓
Container where command is executed

ping db
    ↓
Try to reach the container named "db"
```

`ping` is being used to test network connectivity.

This time it works.

You should see responses similar to:

```
64 bytes from db...
64 bytes from db...
64 bytes from db...
```

The lecture demonstrates that packets are successfully transmitted and received, showing that communication between the containers is working. Pasted text

8. The most important concept

- Containers on the same **Docker Networks** can communicate using **container name** as the **hostname**
- So instead of **localhost** we can refer it as **db** and **pgadmin** using

- -name

9. Dockerized Application

For example, if you have:

```
Spring Boot → PostgreSQL
```

and both are on the same Docker network, your Spring Boot application can use something conceptually like:

```
jdbc:postgresql://db:5432/ecom_db
```

rather than:

```
jdbc:postgresql://localhost:5432/ecom_db
```

Why?

Because from inside the Spring Boot container, `localhost` means **the Spring Boot container itself**, not the PostgreSQL container.

10. Localhost Vs Container Name

This is a common Docker interview question.

Outside Docker

```
localhost:5432
```

means:

> PostgreSQL running on my local machine.

Inside Docker

```
db:5432
```

means:

> Connect to the container whose name is `db` on port 5432.

11. Important Commands

Command	Purpose
`docker run`	Create and run a container
`docker exec`	Execute a command inside a running container
`docker ps`	Show running containers
`docker network create my-network`	Create a Docker network
`docker rm -f db pgadmin`	Force-remove containers
`--network my-network`	Connect container to a network
`docker exec -it pgadmin ping db`	Test communication from pgAdmin to PostgreSQL

12. Complete Story of Demo

Before Docker Network

```
PostgreSQL container
        ❌
pgAdmin container

ping db
   ↓
ERROR
"bad address db"
```

After Docker Network

```
Create network
      ↓
my-network
      ↓
Start PostgreSQL → --network my-network
      ↓
Start pgAdmin    → --network my-network
      ↓
docker exec -it pgadmin ping db
      ↓
SUCCESS ✅
```

13. Interview Answer

Q: Why do we need Docker Networks?

> "Docker networks allow containers to communicate with each other. When multiple containers are connected to the same Docker network, they can communicate using container names as hostnames. This is especially useful in applications where the Spring Boot application, database, cache, and other services run in separate containers."

One line to remember:

> Same Docker Network → Container-to-container communication → Container name can be used as hostname. Pasted text